Deep Agents 实战

第 4 章：任务规划与 LangChain 中间件



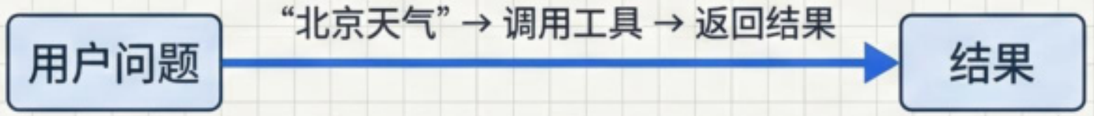
⚠ 规划按需启用 | ⚠ 状态需要校验 | ⚠ 能力按配置组装

Deep Agents 0.7.22

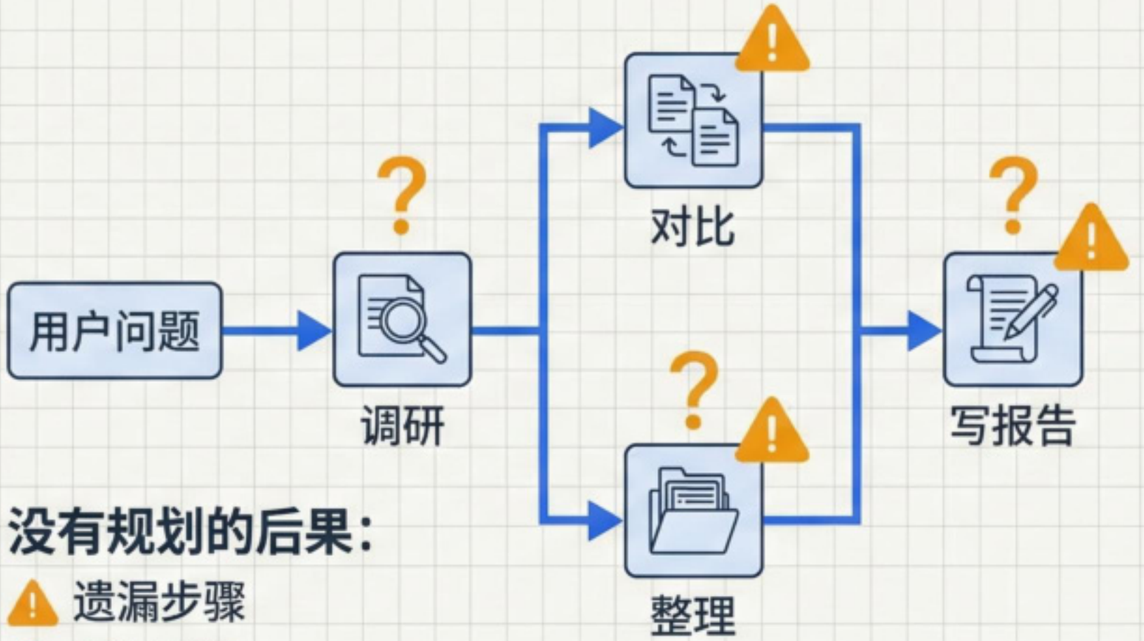
为什么 Agent 需要"规划"能力?

简单任务 vs 复杂任务

简单任务：一步到位



复杂任务：需要多步骤协作



没有规划的后果：

- ⚠ 遗漏步骤
- ⚠ 重复劳动
- ⚠ 半途而废
- ⚠ 质量不稳定

write_todos：两个字段，一份任务清单

任务卡片结构

- **content** — 任务内容（要做的具体事项）
- **status** — 进度标记（任务的当前状态）

示例（JSON）

```
{
  "content": "调研官方文档",
  "status": "pending"
}
```



pending（待办） → in_progress（进行中） → completed（已标记完成）

> ⚠ 这是模型更新的典型流程，不是框架强制状态机；completed 不等于质量验收通过。

已启用后的典型路径：计划、执行、调整

前提：显式启用 TodoListMiddleware

第一步 制定计划

- 搜索资料 [pending]
- 对比方案 [pending]
- 撰写报告 [pending]

第二步 执行与更新

- 搜索资料 [completed]
- 对比方案 [in_progress]
- 撰写报告 [pending]

第三步 动态调整

- 搜索资料 [completed]
- 对比方案 [in_progress]
- 撰写报告 [pending]
- 补充证据 [pending]

调用与顺序由模型决定；同时核对工具轨迹、引用和最终产物。

v0.7：Todo 需要显式启用

示例代码（已配置模型）

```
from deepagents import create_deep_agent
from langchain.agents.middleware import (
    TodoListMiddleware,
)

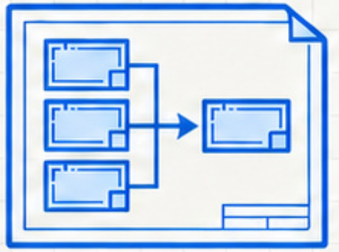
agent = create_deep_agent(
    model=model,
    middleware=[TodoListMiddleware()],
)
```

TodoListMiddleware

- write_todos 工具
- todos 状态
- 规划指导提示词

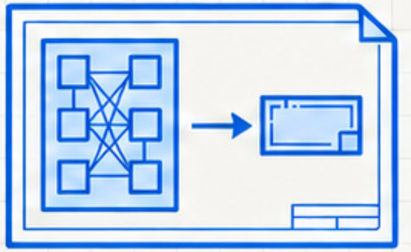
create_agent()

- 需要显式添加 Todo。



create_deep_agent()

- 需要显式添加 Todo。



⚠ 短任务可关闭；长任务与进度 UI 按需启用。

⚠ create_agent 与 create_deep_agent 都需要显式添加 Todo。

中间件有三种配置入口

默认组装

- FilesystemMiddleware
  (7个文件工具 + 权限)
- SummarizationMiddleware
- PatchToolCallsMiddleware
- 模型相关能力依 profile

专用参数

- subagents= → 子 Agent
  (默认含 general-purpose)
- skills= → 技能
- memory= → AGENTS.md
- interrupt_on= → 人工审批

middleware=[...]

- TodoListMiddleware
- Retry / Fallback
- PII / 调用次数限制

同名实例在原位置完整替换，不会逐字段合并配置。

checkpointer= 属于 LangGraph 持久化，不是中间件。

分组是配置方式，不代表 Hook 执行顺序。

两类 Hook：节点边界 vs 调用包装

节点边界（Node-style）

独立图节点

before_agent → before_model → model → after_model → after_agent

生命周期示意，省略工具循环

状态更新、校验、审计

调用包装（Wrap-style）

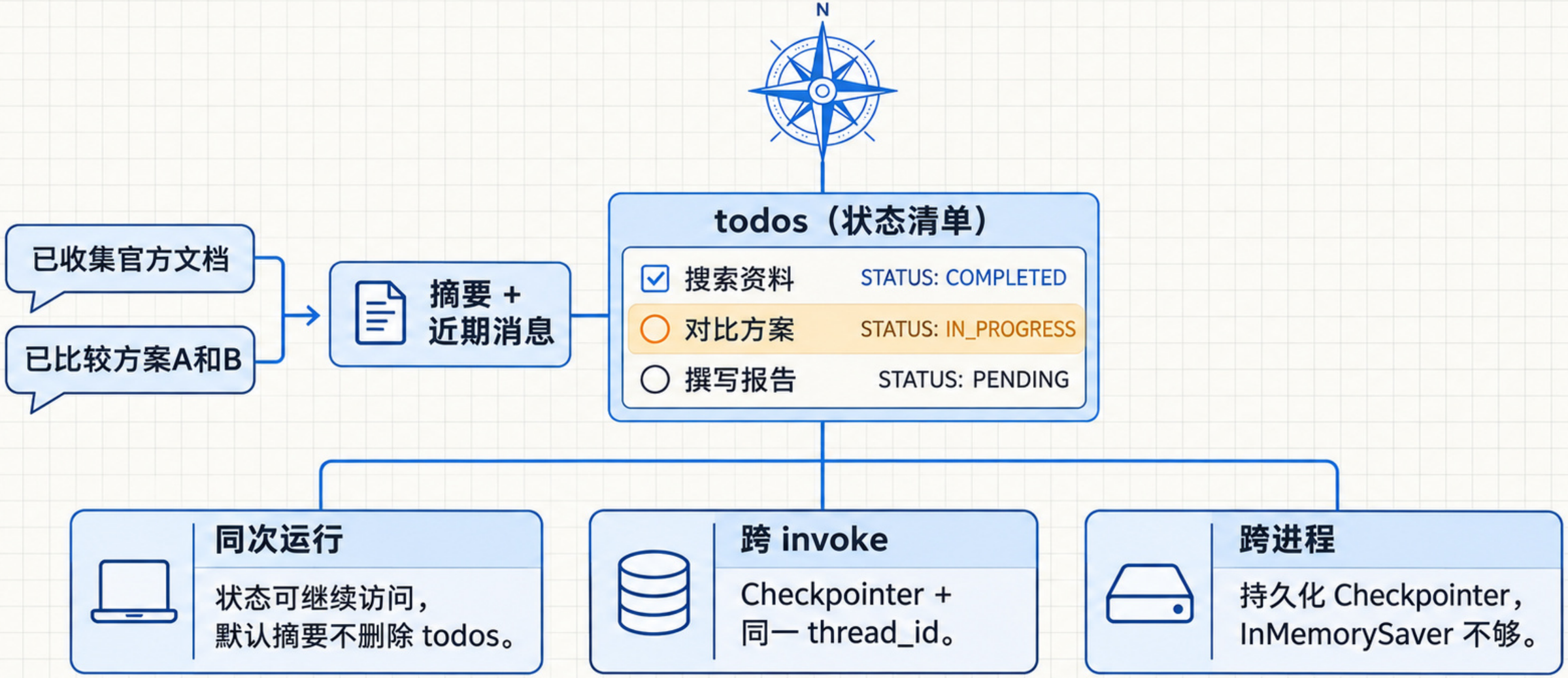


缓存、重试、降级

⚠ 中断优先放在 Node-style 边界；恢复会重放节点，副作用需幂等。

⚠ Wrap 位于节点内部，恢复可能连同 handler 重跑。

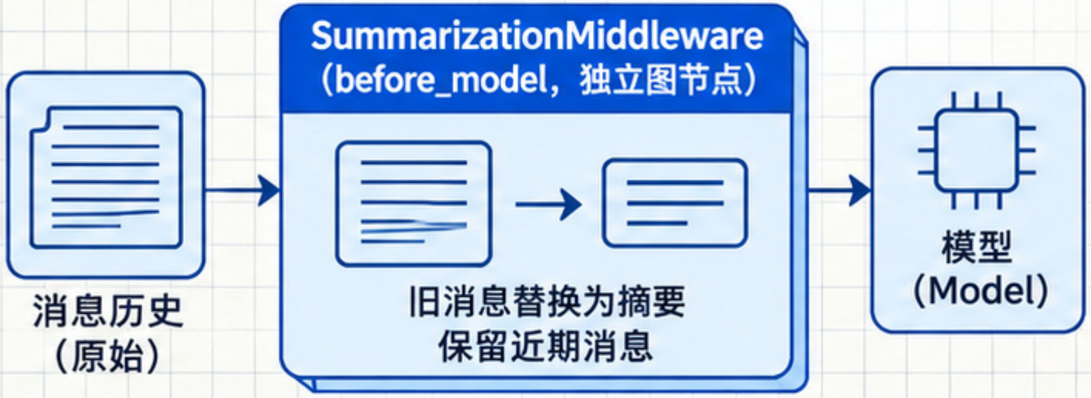
todos 与消息分开保存，不等于自动永久保存



- 子 Agent 默认不继承主 Agent 的 Todo 中间件；
- 隔离模式下清单不自动读写回主 Agent。

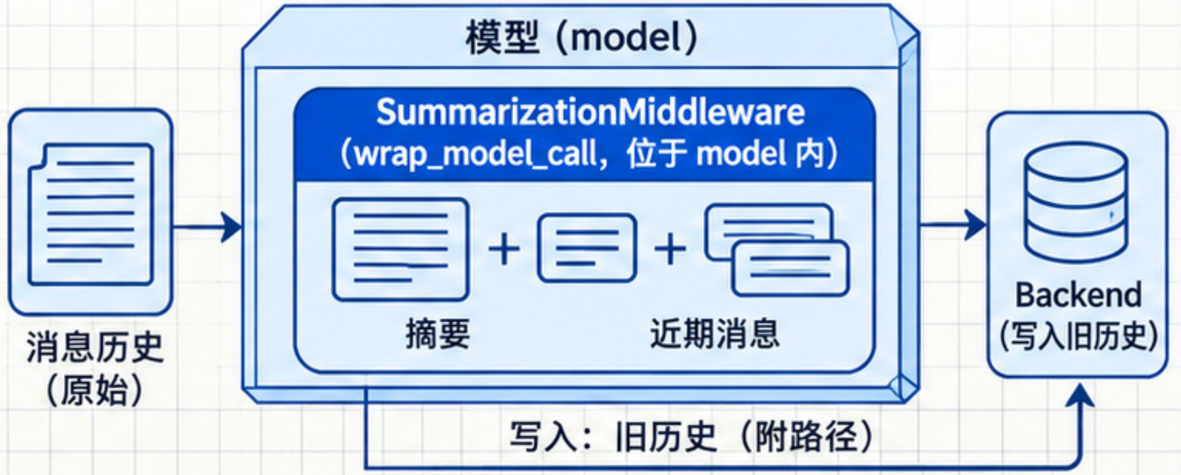
同名 SummarizationMiddleware，行为不同

LangChain / langchain.agents.middleware



- before_model 独立图节点
- 旧消息替换为摘要，保留近期消息
- 不负责写入 Backend

Deep Agents / deepagents.middleware



- wrap_model_call 位于 model 内
- 原始消息保留，另存摘要与截断位置
- 模型输入为摘要 + 近期消息
- 旧历史写入 Backend 并附路径

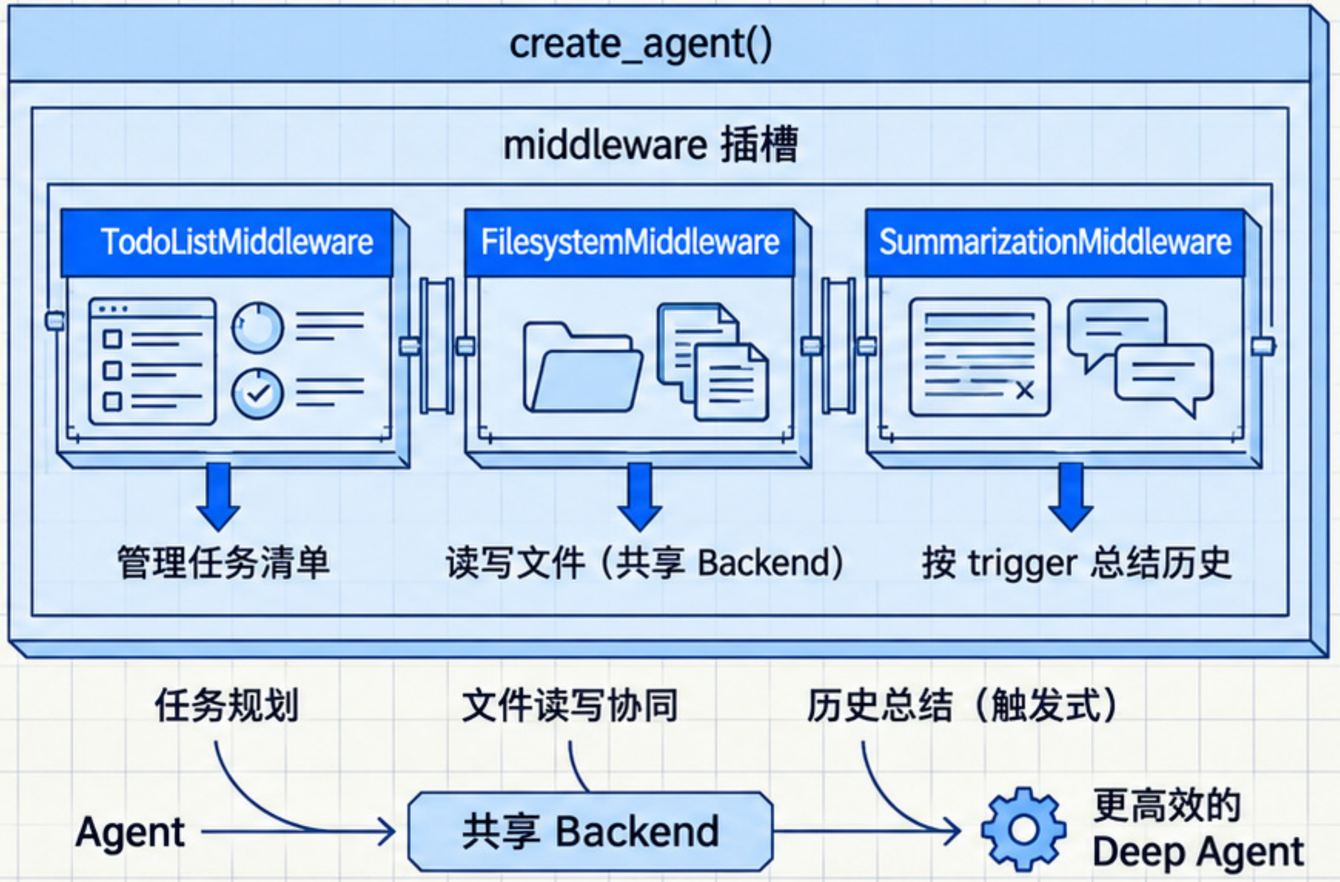
⚠ 手动实例化时 trigger 默认 None，不会按阈值主动摘要；create_deep_agent 会配置默认触发条件。

ⓘ 图中没有独立摘要节点，不代表没有摘要逻辑。溢出异常处理是另一条触发路径。

手动组合：共享 Backend + 明确 trigger

```python
from langchain.agents import create_agent
from langchain.agents.middleware import TodoListMiddleware
from deepagents.backends import StateBackend
from deepagents.middleware import (
    FilesystemMiddleware, SummarizationMiddleware,
)

backend = StateBackend()
agent = create_agent(model=model, middleware=[
    TodoListMiddleware(),
    FilesystemMiddleware(backend=backend),
    SummarizationMiddleware(
        model=model, backend=backend,
        trigger=("tokens", 4000), keep=("messages", 20),
    ),
])
```



⚠ model 需预先配置；4000/20 仅为示例。共享 Backend 让 read_file 可读回历史；无可总结旧消息时不生成摘要。

示意片段・非完整 Harness 复刻

修补消息 ≠ 重试工具 ≠ 恢复状态

PatchToolCallsMiddleware →
缺少对应 ToolMessage
→ 补取消或未执行说明
before_agent
不重试、不证明
外部副作用已撤销

ToolRetryMiddleware →
可重试异常
→ 按显式策略重试
写操作关注幂等

Checkpointer + thread_id →
保存/恢复状态
跨进程需持久化
不是从任意
代码行续跑

校验与评测 →
轨迹 + 引用 + 产物 + 测试
→ 判断结果质量

示例：消息修补时间线

call_1 请求 → 响应缺失 → 补同一 tool_call_id 的说明

⚠ 已有错误响应不属于缺失响应；调用成功也不等于结果正确。

本章回顾



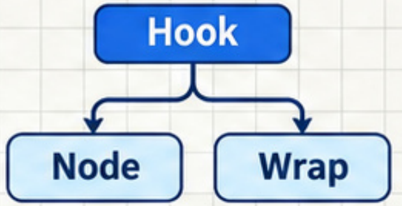
Todo 显式启用：
工具 + todos + 提示词



content/status：
进度标记不等于验收

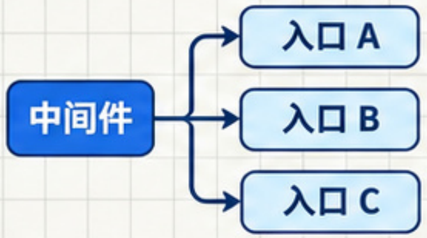


Checkpointer：
按线程接续，持久化看实现

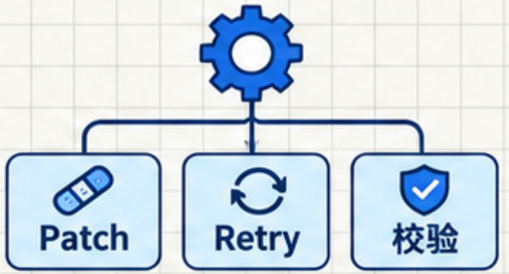


Hook 分 Node 与 Wrap：

⚠ 注意重放边界

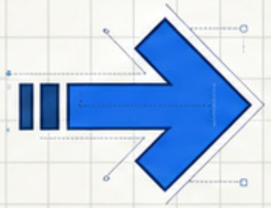


中间件三入口：
同名实例完整替换



Patch / Retry / 校验：

⚠ 职责不可混淆



第 5 章：子 Agent 与上下文隔离

核对基线：Deep Agents 0.7.22 / LangChain 1.4.3